DBT TRAINING

Working Effectively · Module 09 · 90 min

Jinja and Macros

Stop copying the same CAST expression into every model. Start writing reusable SQL logic — and know exactly when not to.

An error occurred on this slide. Check the terminal for more information.

The Problem

You wrote this CAST in 12 different staging models

`stg_hubspot__deals.sql`

```
SELECT
  deal_id,
  CAST(amount AS DOUBLE)    AS amount,
  CAST(close_date AS DATE)  AS expected_close_date
FROM {{ source('hubspot', 'deals') }}
```

`stg_hubspot__contacts.sql`

```
SELECT
  contact_id,
  CAST(score AS INTEGER)    AS contact_score,
  CAST(created_at AS DATE)  AS created_date
FROM {{ source('hubspot', 'contacts') }}
```

The hidden cost

Source data from Bronze is messy. When `amount` contains `'N/A'`, `CAST` throws a runtime error and fails the entire model run. You need `TRY_CAST` — but now you have to fix it in 12 places. And next time someone adds a 13th model, they'll use `CAST` again because there's no convention to follow.

Jinja Syntax — The Three Delimiters

A macro uses all three. Know which does what.

Delimiter	Purpose	Renders output?	Example
`{{ }}`	Expression — evaluates and outputs a value	Yes	`{{ ref('fct_deal') }}` · `{{ my_macro(arg) }}`
`{% %}`	Statement — logic, control flow, definitions	No	`{% if is_incremental() %}` · `{% macro name(args) %}`
`{# #}`	Comment — ignored entirely	No	`{# TODO: add null guard #}`

Wrong — breaks immediately

```
{{ macro safe_cast(col, type) }}
    TRY_CAST({{ col }} AS {{ type }})
```

Correct

```
{% macro safe_cast(col, type) %}
    TRY_CAST({{ col }} AS {{ type }})
```

Your First Macro: `safe_cast`

macros/safe_cast.sql — definition

```
{% macro safe_cast(
    column_name,
    target_type,
    fallback=None
) %}
{%- if fallback is not none -%}
  COALESCE(
    TRY_CAST({{ column_name }}
             AS {{ target_type }}
    {{ fallback }}
  )
{%- else -%}
  TRY_CAST({{ column_name }}
           AS {{ target_type }})
{%- endif -%}
{% endmacro %}
```

stg_hubspot__deals.sql — call

```
SELECT
  deal_id,
  {{ safe_cast(
       'amount',
       'DOUBLE'
     ) }}              AS amount,
  {{ safe_cast(
       'close_date',
       'DATE'
     ) }}              AS expecte
  {{ safe_cast(
       'deal_score',
       'INTEGER',
       '0'
     ) }}              AS deal_sc
FROM {{ source('hubspot', 'deals'
```

target/compiled/... — SQL sent to DuckDB

```
SELECT
  deal_id,
  TRY_CAST(amount AS DOUBLE)
    AS amount,
  TRY_CAST(close_date AS DATE)
    AS expected_close_date,
  COALESCE(
    TRY_CAST(deal_score AS INTEGE
    0
  )                      AS deal_sc
FROM main.raw_deals
```

No Jinja in the output. DuckDB receives plain SQL.

`dbt_utils.generate_surrogate_key`

Why surrogate keys? Where to generate them?

Why a surrogate key?

Natural keys (HubSpot IDs) are strings that can change, merge, or be reused.

A hash key is stable, join-safe across systems, and works as a consistent dimension key in Power BI.

Every Silver model needs one. Without it, joins to fact tables are fragile.

Where to call it

In the **source CTE** of each model — not in a shared macro. Surrogate key generation is part of the row's identity. Put it where the row is first selected.

models/silver/fct_deal.sql

```
WITH source AS (
    SELECT
        {{ dbt_utils.generate_surrogate_key(
               ['deal_id']
           ) }}          AS deal_key,
        deal_id,
        deal_name,
        pipeline_id,
        amount,
        expected_close_date,
        ingested_at
    FROM {{ ref('stg_hubspot__deals') }}
)

SELECT * FROM source
```

Compiles to: `MD5(CAST(deal_id AS TEXT)) AS deal_key`

The Macro Antipattern

What happens when business logic moves into Jinja

`macros/classify_deal.sql — do not write this`

```
{% macro classify_deal(amount_col, stage_col) %}
  CASE
    WHEN {{ stage_col }} = 'closed-won'
         AND {{ amount_col }} > 50000
      THEN 'enterprise-closed'
    WHEN {{ stage_col }} = 'closed-won'
         AND {{ amount_col }} > 10000
      THEN 'mid-market-closed'
    WHEN {{ stage_col }} IN ('proposal','demo')
         AND {{ amount_col }} > 50000
      THEN 'enterprise-pipeline'
    -- ... 20 more lines of business rules
  END
{% endmacro %}
```

- ❌ Business rules are invisible to SQL analysts
- ❌ Cannot write an `accepted_values` test targeting this logic
- ❌ A BI developer reading `fct_deal` cannot understand what `deal_segment` means without reading the macro file

`models/silver/fct_deal.sql — write this instead`

```
WITH classified AS (
    SELECT
        deal_id,
        amount,
        pipeline_stage,
        CASE
          WHEN pipeline_stage = 'closed-won'
               AND amount > 50000
            THEN 'enterprise-closed'
          WHEN pipeline_stage = 'closed-won'
               AND amount > 10000
            THEN 'mid-market-closed'
          WHEN pipeline_stage IN ('proposal','demo')
               AND amount > 50000
            THEN 'enterprise-pipeline'
          ELSE 'other'
        END AS deal_segment
    FROM {{ ref('stg_hubspot__deals') }}
)
```

When to Use What

Decision table: macro vs. CTE vs. ephemeral

Situation	Right tool	Reason
Same `TRY_CAST` pattern in 8+ staging models	Macro	Structural — purely mechanical, no business meaning
Surrogate key hash over a column	`dbt_utils.generate_surrogate_key`	Already a package macro — call it directly
`CASE WHEN` mapping pipeline stages to categories	SQL CTE in the model	Business logic — visible, reviewable, testable
Intermediate join used by exactly one model	SQL CTE or ephemeral	Single use — abstraction adds complexity, no benefit

Hooks

Run SQL before or after a model executes

dbt_project.yml — project-level hooks

```yaml
models:
  analytics:
    silver:
      facts:
        +post-hook:
          - >
            ALTER TABLE {{ this }}
            ADD PRIMARY KEY (deal_key)
            NOT ENFORCED
```

model config — inline hook

```
{{ config(
    materialized = 'table',
    post_hook    = [
      "GRANT SELECT ON {{ this }}
       TO ROLE REPORTER"
    ]
) }}
```

pre-hook

Runs before the model SQL. Use for: drop temp tables, set session variables, write an audit log entry.

post-hook

Runs after the model SQL. Use for: add constraints, grant permissions, write a completion audit log entry.

Hooks are powerful and hard to debug. Prefer solving the problem in dbt config before reaching for hooks.

Packages

Install pre-built macro libraries with `packages.yml`

`packages.yml (project root)`

```yaml
packages:
  - package: dbt-labs/dbt_utils
    version: 1.3.0
  - package: calogica/dbt_expectations
    version: 0.10.4
```

After adding or updating `packages.yml`, run:

```
dbt deps
```

Downloads to `dbt_packages/` — gitignored. Run `dbt deps` after every clone.

dbt_utils macros used in this project

`generate_surrogate_key([cols])` — MD5 hash surrogate key

`star(from=ref('model'), except=[...])` — SELECT * except columns

`pivot(col, values, ...)` — pivot a column into boolean columns

dbt_expectations tests used in this project

`expect_column_values_to_be_between` — numeric range check

`expect_column_pair_values_to_be_equal` — assert two columns match

KEY TAKEAWAYS

Macros compile to plain SQL. Jinja is an authoring tool — DuckDB and Snowflake never see it. When in doubt, run `dbt compile` and read the output.

Use macros for structural patterns, not business logic. Type coercion and key generation belong in macros. CASE WHEN logic that classifies deals belongs in SQL — where it's visible, testable, and reviewable.

Call `dbt deps` after every clone. `dbt_packages/` is gitignored. If `generate_surrogate_key` isn't found, you haven't run `dbt deps`.

MODULE 09 COMPLETE

Next: Module 10

Snapshots and Slowly Changing Dimensions (SCD2)

```
Prep Q1: What is a slowly changing dimension? What type
does dbt snapshots implement by default?
```

```
Prep Q2: If a contact's email changes in HubSpot, how
would you preserve both the old and new value with
history?
```

```
Prep Q3: What columns does a dbt snapshot add to track
row history?
```

```
Prep Q4: In dim_contact, contact_key is a hash of
hubspot_contact_id. If a contact's email changes, does
contact_key change? Why does that matter for snapshot
strategy?
```